

autolisp-reader Usage Guide

Codex

May 25, 2026

Contents

1	Purpose	1
2	Package	1
3	Main Entry Points	1
3.1	1. Tokenization	2
3.2	2. Form Reading	2
3.3	3. Concrete Comment-Preserving Reading	3
4	Read Results	3
5	Modes And Options	4
5.1	Token Acceptance Mode	4
5.2	Reader Behavior Options	5
6	Implemented Options	6
6.1	:token-mode	6
6.2	:retain-comments-p	7
6.3	:extended-string-escapes-p	7
6.4	:warn-on-integer-overflow-p	8
6.5	:canonical-case	8
6.6	:source-name	8
6.7	:recover-malformed-p	9
7	Numeric Behavior	9
7.1	Integer Range	9
7.2	Overflow Warning	9
7.3	Strict Versus Lax Reals	9

8 Object Families	10
9 Typical Workflows	11
9.1 Normal Reader Use	11
9.2 Tooling / Formatter Use	11
9.3 Lexical Inspection	11
9.4 Reading From A String With Source Metadata	11
10 Current Limitations	12
11 Related Files	12

1 Purpose

This document explains how to use the current `autolisp-reader` implementation.

It is practical API documentation, not a normative syntax specification. The normative and design-oriented material remains in:

- documentation/specification.org

2 Package

The public API lives in the package:

```
clautolisp.autolisp-reader
```

Typical use:

```
(require :asdf)
(asdf:load-system "clautolisp/autolisp-reader")
(use-package :clautolisp.autolisp-reader)
```

3 Main Entry Points

There are three families of entry points.

3.1 1. Tokenization

Use tokenization when you want the lexical token stream.

- `tokenize-string`
- `tokenize-stream`
- `tokenize-file`

Example:

```
(defparameter *result*  
  (tokenize-string "(foo 17 3.5)"))  
  
(mapcar #'token-kind (read-result-objects *result*))  
;; => (:LEFT-PAREN :SYMBOL :INTEGER :REAL :RIGHT-PAREN :EOF)
```

3.2 2. Form Reading

Use form reading when you want parsed reader objects suitable for normal language processing.

- `read-forms-from-string`
- `read-forms-from-stream`
- `read-forms-from-file`

Comments are skipped in this mode.

Example:

```
(defparameter *result*  
  (read-forms-from-string "(a . c)"))  
  
(defparameter *object* (first (read-result-objects *result*)))  
  
(cons-object-dotted-p *object*)  
;; => T
```

3.3 3. Concrete Comment-Preserving Reading

Use concrete reading when you want comments and dot markers preserved explicitly for tooling.

- `read-concrete-from-string`
- `read-concrete-from-stream`
- `read-concrete-from-file`

This mode is intended for:

- formatters,
- source-to-source transforms,
- editors,
- structural source inspection.

Example:

```
(defparameter *result*  
  (read-concrete-from-string  
    "( a ; c1  
    . ; c2  
    c ; c3  
    )"))  
  
(type-of (first (read-result-objects *result*)))  
;; => CONCRETE-LIST-OBJECT
```

4 Read Results

All public entry points return a `read-result` object.

Use:

- `read-result-objects`
- `read-result-diagnostics`

Example:

```
(let ((result (tokenize-string "2147483648"
                               :warn-on-integer-overflow-p t)))
      (values
        (read-result-objects result)
        (read-result-diagnostics result)))
```

The result model separates:

- returned objects or tokens,
- diagnostics such as warnings.

5 Modes And Options

The current implementation exposes two distinct ideas that are easy to confuse:

1. token acceptance mode,
2. reader behavior options.

5.1 Token Acceptance Mode

The token acceptance mode is controlled by:

- `:token-mode :strict`
- `:token-mode :lax`

It is accepted by all main entry points.

5.1.1 Strict

`:strict` accepts only the conservative token syntax supported by the current implementation baseline.

Notable examples:

- integer tokens like `17` are accepted,
- real tokens like `3.5` are accepted,
- `1e3` is *not* treated as a real token in strict mode,

- `.5` is *not* treated as a real token in strict mode.

Example:

```
(token-kind
 (first
  (read-result-objects
   (tokenize-string "1e3" :token-mode :strict))))
;; => :SYMBOL
```

5.1.2 Lax

`:lax` admits broader real-number token syntax.

In the current implementation this includes forms such as:

- `1e3`
- `.5`
- `1.`

Example:

```
(token-kind
 (first
  (read-result-objects
   (tokenize-string "1e3" :token-mode :lax))))
;; => :REAL
```

Portable syntax is accepted in both modes. Lax mode only broadens token acceptance beyond that common core.

5.2 Reader Behavior Options

Reader behavior is controlled separately from strict versus lax token acceptance.

The public constructor is:

```
(make-reader-options ...)
```

Public accessors:

- `reader-options-token-mode`

- `reader-options-retain-comments-p`
- `reader-options-recover-malformed-p`
- `reader-options-warn-on-integer-overflow-p`
- `reader-options-extended-string-escapes-p`
- `reader-options-canonical-case`
- `reader-options-source-name`

You may either:

- pass individual keyword arguments directly to the reading functions,
- or build a `reader-options` object and pass it as `:options`.

Example:

```
(defparameter *options*
  (make-reader-options
    :token-mode :lax
    :warn-on-integer-overflow-p t
    :extended-string-escapes-p t
    :canonical-case :upcase
    :source-name "example.lsp"))

(read-forms-from-string "(foo 2147483648)"
  :options *options*)
```

6 Implemented Options

6.1 `:token-mode`

Accepted values:

- `:strict`
- `:lax`

This controls lexical acceptance only.

6.2 `:retain-comments-p`

Controls whether comments are emitted by tokenization.

- for `tokenize-*`, it determines whether comment tokens appear in the token stream,
- for `read-forms-*`, comments are always skipped,
- for `read-concrete-*`, comments are always preserved in the concrete syntax result.

Therefore this option matters most for direct tokenization.

6.3 `:extended-string-escapes-p`

Default: `NIL`

This option controls implementation-defined backslash escape handling inside string literals.

When false, the reader stays conservative and only guarantees the portable string behavior already documented for strict reading, namely doubled double quotes plus the default baseline escapes `\"` and `\\`.

When true, the reader additionally accepts:

- `\u+00E9` style Unicode escapes using exactly four hexadecimal digits,
- backslash before another character to quote that character literally.

This option is separate from external-format decoding.

- External format controls how bytes become characters before tokenization.
- Extended string escapes control how ASCII escape syntax inside a string is interpreted by the reader.

Example:

```
(mapcar #'token-kind
  (read-result-objects
    (tokenize-string "a ; c
b"
                    :retain-comments-p t)))
;; => (:SYMBOL :COMMENT :SYMBOL :EOF)
```


6.4 `:warn-on-integer-overflow-p`

When true, the reader emits a warning diagnostic if an integer-shaped decimal token is read as a real because it falls outside the signed 32-bit integer range.

Example:

```
(let ((result (tokenize-string "2147483648"
                                :warn-on-integer-overflow-p t)))
      (mapcar #'diagnostic-code
              (read-result-diagnostics result)))
;; => (:INTEGER-OVERFLOW-READ-AS-REAL)
```

6.5 `:canonical-case`

Controls how symbol names are canonicalized.

Accepted values in the current implementation:

- `:upcase`
- `:downcase`
- `:preserve`

The original token spelling remains available separately.

Example:

```
(let* ((result (read-forms-from-string "Foo"
                                         :options
                                         (make-reader-options
                                          :canonical-case :downcase)))
        (object (first (read-result-objects result))))
      (values (symbol-object-original-name object)
              (symbol-object-canonical-name object)))
;; => "Foo", "foo"
```

6.6 `:source-name`

Sets the source designator recorded in source spans.

This is useful when reading from strings or custom streams and still wanting useful span metadata.

6.7 `:recover-malformed-p`

This option exists in the public options structure, but malformed-structure recovery is not yet implemented in the current reader.

At present, malformed list or dotted-pair syntax still signals an error rather than producing a recoverable syntax object stream.

7 Numeric Behavior

7.1 Integer Range

The current implementation classifies decimal integer-shaped tokens as integers only when the numeric value lies in:

- `[-2147483648, 2147483647]`

Outside that range, the token is read as a real object.

Example:

```
(type-of
 (first
  (read-result-objects
   (read-forms-from-string "2147483648"))))
;; => REAL-OBJECT
```

7.2 Overflow Warning

Overflow reclassification does not fail the read. It may emit a warning diagnostic if requested.

7.3 Strict Versus Lax Reals

Current examples:

- strict accepts 3.5
- lax additionally accepts 1e3, .5, and 1.

8 Object Families

The public API exposes typed reader-layer objects.

Important families include:

- `token`
- `symbol-object`
- `string-object`
- `integer-object`
- `real-object`
- `cons-object`
- `quote-object`
- `comment-object`
- `dot-object`
- `concrete-list-object`
- `diagnostic`
- `source-span`

The accessor names are intentionally direct:

- `symbol-object-original-name`
- `symbol-object-canonical-name`
- `string-object-value`
- `integer-object-value`
- `real-object-value`
- `cons-object-elements`
- `cons-object-tail`
- `quote-object-object`

- `comment-object-text`
- `token-kind`
- `token-lexeme`
- `diagnostic-code`
- `diagnostic-message`

9 Typical Workflows

9.1 Normal Reader Use

Use `read-forms-from-*`.

```
(read-forms-from-file #P"test.lsp"
                      :token-mode :strict)
```

9.2 Tooling / Formatter Use

Use `read-concrete-from-*`.

```
(read-concrete-from-file #P"test.lsp"
                        :token-mode :strict)
```

9.3 Lexical Inspection

Use `tokenize-*`.

```
(tokenize-file #P"test.lsp"
               :token-mode :lax
               :retain-comments-p t)
```

9.4 Reading From A String With Source Metadata

```
(read-forms-from-string "(foo 17)"
                       :source-name "inline-example")
```

10 Current Limitations

The following are important current limitations of the implementation as it exists now:

- malformed-structure recovery is not yet implemented, even though the options structure reserves a slot for it,
- the concrete comment-preserving mode is a separate concrete syntax view, not a full round-tripping printer,
- the strict and lax numeric policies are implemented only for the currently supported token classes,
- the string escape repertoire is intentionally conservative,
- this guide documents the current implementation, which is still an early module and not the final project-wide reader contract.

11 Related Files

- `README.org`
- `PLAN.md`
- `documentation/specification.org`